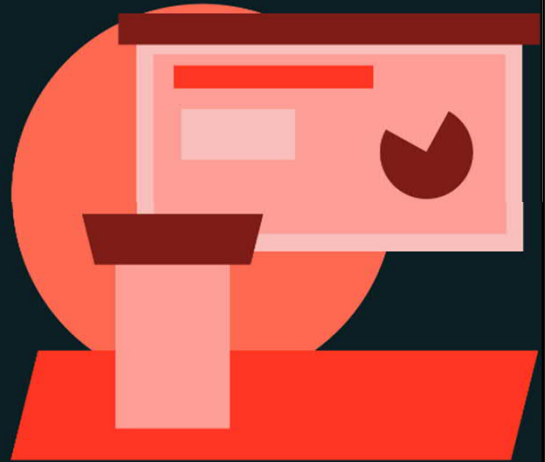




Introduction to Apache Spark™

LECTURE

Apache Spark™ Runtime Architecture



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

This lecture provides an in-depth overview of the Apache Spark™ Runtime Architecture, its core components, execution flow, and how Spark operates within Databricks clusters.

Lesson Learning Objectives

- Identify the roles of the Driver, Workers, and Executors in Spark applications.
- Understand the role of the Spark Driver and learn how SparkSession serves as the unified entry point for Spark applications.
- Understand how Worker nodes execute tasks and how resources are allocated within Spark applications.
- Utilize the Spark UI to monitor application progress, DAG visualizations, and resource allocation.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

By the end of this lecture, you will be able to describe the runtime architecture and different roles within a Spark application, including the driver, workers, and executors; explain the Spark driver and the Spark session as the entry point; understand executors and resource management; describe how tasks and work are distributed; and visualize a Spark application at runtime using the Spark UI.

What is Apache Spark?

Unified Large-Scale Data Processing and Analytics Framework

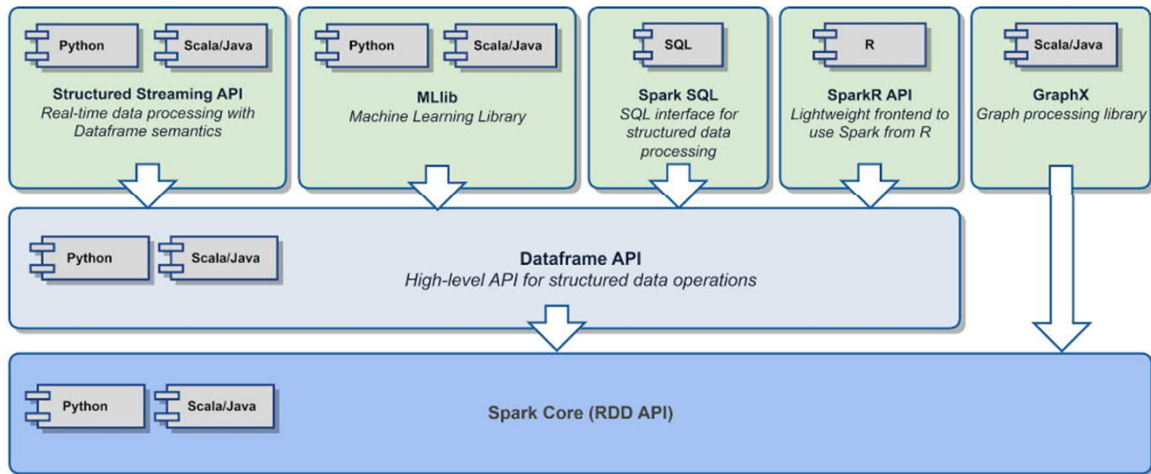
- Apache Spark is an open-source, distributed, unified analytics engine for fast, large-scale data processing
- Features native support for SQL, streaming data, machine learning, and graph processing through a consistent API
- Delivers high performance through in-memory computation, making it significantly faster than other processing frameworks
- Operates seamlessly with diverse data sources including cloud storage (AWS S3, Azure, Google), local files, and databases



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Apache Spark is a unified analytics framework providing a consistent interface for handling big data across multiple domains. It is open-source and distributed, built for large-scale processing and horizontal scalability. It has native support for multiple interfaces, including SQL, streaming, machine learning, graph processing, and more. It is designed for high performance, built around in-memory processing, which is significantly faster than disk-based systems that preceded it. It seamlessly integrates into cloud environments, enabling reading and writing to and from cloud storage.

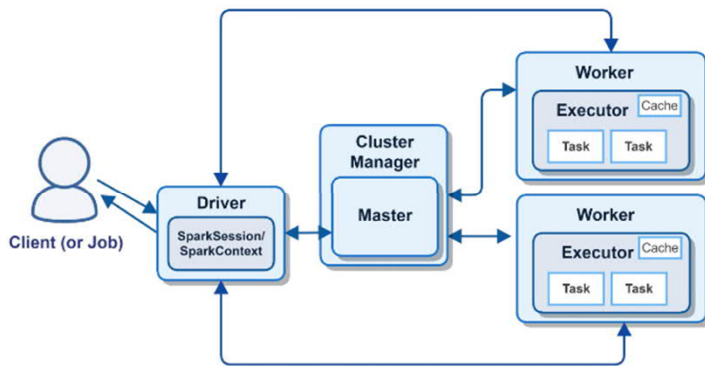
Spark Components



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

The Spark Core Engine provides the foundation for all Spark applications, handling memory management, fault recovery, scheduling, and task distribution. It is exposed through high-level APIs such as the DataFrame API, RDDAPI, and SQL API, which are available across multiple languages including Python, Scala, Java, and R. Specialized libraries built on top of the core engine, such as Spark SQL, MLlib, GraphX, and Structured Streaming, provide advanced functionalities for various use cases.

Apache Spark Runtime Architecture



- **Driver:** The brain of a Spark application, responsible for planning and coordinating execution.
- **Cluster Manager/Master:** Manages cluster resources and allocates them to the Driver (internal).
- **Workers:** Nodes in the cluster that host Executors.
- **Executors:** Processes on Worker nodes that execute tasks assigned by the Driver.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

The Driver is the central component that manages the application lifecycle and execution. The Cluster Manager (Master) allocates resources and assigns tasks to workers; on Spark/Databricks clusters. Workers perform the tasks assigned by the driver, execute code, and return results. Executors run on worker nodes, handling task execution and caching data.

The Spark Driver

Planning, Coordination, and Task Execution Management

- Creates the **SparkSession**, the entry point for all Spark applications
- Analyzes the Spark application and constructs a Directed Acyclic Graph (DAG)
- Schedules and distributes tasks to Executors for execution
- Monitors the progress of tasks and handles failures
- Returns results to the client



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Manages the application lifecycle by creating the SparkSession for communication with the cluster. Analyzes jobs by constructing a Directed Acyclic Graph (DAG) for processing. Schedules tasks by distributing work to Executors. Monitors progress and handles failures to ensure fault tolerance and efficient resource use. Returns results to the client, ensuring end-to-end data processing.

SparkSession

Spark Application Entry Point

- Introduced in Spark 2.0 as a unified entry point for all contexts (formerly instantiated individually as **SparkContext**, **SQLContext**, **HiveContext**, **StreamingContext**)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MySparkApplication") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()
```

- Created for you automatically in Databricks as **spark**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Unified entry point introduced in Spark 2.0 as a consolidation of various contexts. Previously, users had to manage separate contexts like SparkContext, SQLContext, and HiveContext. Now, SparkSession encapsulates all functionalities. In Databricks, it is automatically created as spark.

Executors

Task Execution and I/O

- Run on Worker nodes in a Spark cluster and host **Tasks** (for I/O and data processing), each Worker node can run multiple Executors, with the number depending on:
 - Available CPU cores (`spark.executor.cores`)
 - Memory resources (`spark.executor.memory`)
 - Configuration settings
- Store intermediate and final results in memory or on disk
- Interact with the Driver for task coordination and data transfer



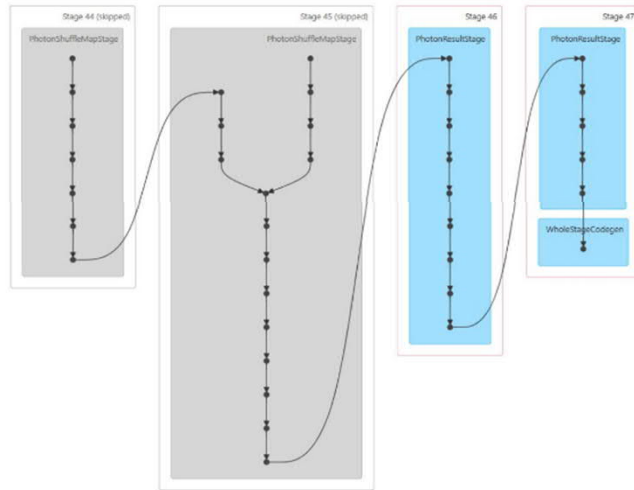
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Task execution and I/O handle data processing tasks and communicate results back to the driver. Executors run on worker nodes, with each node capable of hosting multiple executors. Resource management is configured through settings like `spark.executor.cores` and `spark.executor.memory`. Executors can store intermediate and final results in persistent storage. They interact with the driver by processing assigned tasks and reporting status.

The Spark DAG

Directed Acyclic Graph

- Spark jobs are broken down into **Stages** – groups of tasks that can be executed in parallel
- Computations flow in one direction through the stages (the **Directed** bit)
- Stages never loop back, ensuring the job terminates (the **Acyclic** bit)
- Stages are organized into a dependency graph for execution flow (the **Graph** bit)



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Stages are groups of tasks that can be executed in parallel. Operations flow in a directed manner from one stage to the next, with no backward loops. This acyclic structure ensures computations complete successfully without infinite loops. A dependency graph is used to plan the execution flow for parallel processing.

1. What does "DAG" actually mean?

Directed: The process flows in one direction (from input data to final result).

Acyclic: There are no loops. You can't go back to a previous state once a transformation is done.

Graph: It's a mathematical structure of **Vertices** (the data/RDDs) and **Edges** (the operations like filter or map).

2. How the DAG is Built (The Lifecycle)

Spark doesn't run your code line-by-line. It uses **Lazy Evaluation**.

Transformations: When you write `df.filter()` or `df.select()`, Spark simply adds a "note"

to the DAG. No data is moved yet.

Action: Only when you call an **Action** (like `.show()`, `.count()`, or `.save()`) does Spark look at the entire DAG.

Optimization: Spark's optimizer (Catalyst) looks for ways to combine steps (e.g., merging two filters into one) to save time.

Submission: The DAG is handed to the **DAG Scheduler**, which breaks it into **Stages** and **Tasks**.

3. The Hierarchy: Job → Stage → Task

To understand how a DAG is executed, you need to see how it's sliced:

A. Jobs

Every time you call an **Action**, a new **Job** is created. If your script has three `.show()` calls, you will see three Jobs in the Spark UI.

B. Stages

Spark divides a Job into Stages based on **Shuffle Boundaries**.

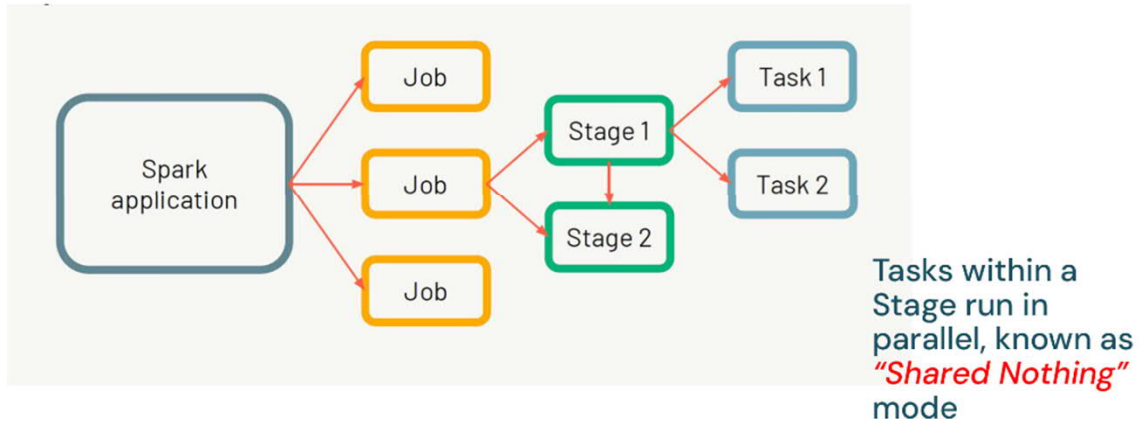
Narrow Transformations: (e.g., `map`, `filter`) Operations that happen on a single partition of data stay in the **same stage**.

Wide Transformations: (e.g., `groupBy`, `join`) These require data to be "shuffled" across the network to different nodes. This **starts a new stage**.

C. Tasks

The smallest unit of work. Each stage is divided into **Tasks** based on the number of data partitions. If a stage has 10 partitions, Spark creates 10 tasks to run in parallel.

Spark Application Execution



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Jobs are high-level operations triggered by actions in Spark, such as `collect()` or `save()`. Stages divide jobs into smaller, independent units that can run in parallel. Tasks are the individual units of work executed by Executors. In parallel execution, tasks within a stage run independently using a "Shared Nothing" architecture.

The Spark UI

Visualising Spark Applications

Spark provides web user interfaces for monitoring and management, including:

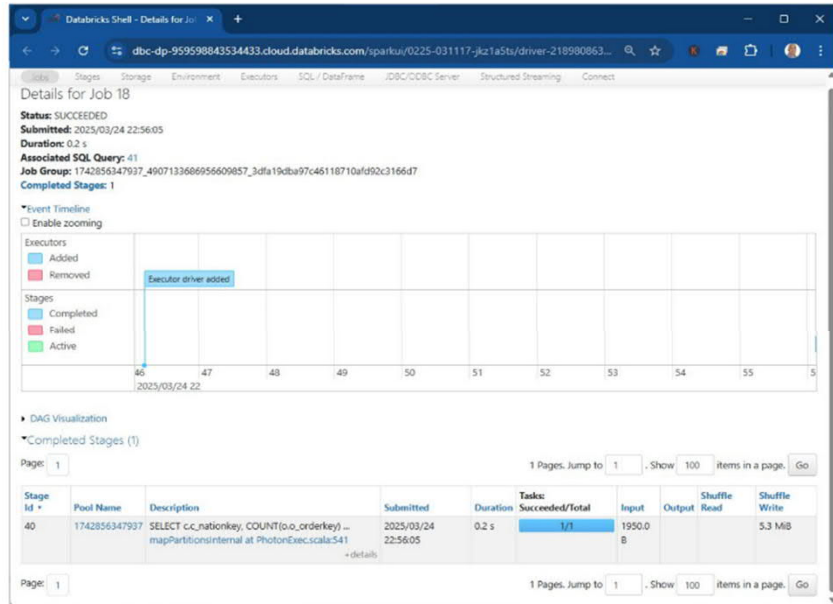
- **Application UI** – per application (**SparkSession**)
 - Application progress and task execution
 - DAG visualization and stage details
 - Resource usage and performance metrics
- **Master UI** – per cluster
 - Worker node status and health and cluster-wide resource allocation
 - Shows all running applications and available resources



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Spark provides a user interface (UI) that allows you to monitor the Spark application, including metrics and visualizations of the DAG, resource usage, application progress, and more. There are two user interfaces: the Application UI, which is per application and monitored throughout the Spark session, and the Master UI, which provides a clusterwide view of node health and resource allocation. Within the Databricks platform, these can be accessed during a demo. The Spark UI is particularly useful for troubleshooting and optimizing performance, identifying bottlenecks, debugging applications interactively, and more.

The Spark UI



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

The Application UI provides per-application monitoring through the SparkSession, offering details on progress, DAG visualization, resource usage, and more. The Master UI gives a cluster-wide view for monitoring multiple applications, showing the health status of nodes and overall resource allocation. Both interfaces are useful for interactive debugging, troubleshooting, and optimizing performance.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).